SageMath: A Closer Examination of Graphs

In the second article in this series on SageMath (published in the February 2024 issue of *OSFY)*, we used this software system to create and extract information from graphs. We also discussed how undirected simple graphs can be created using SageMath. In this third article in the series, we will continue exploring graph theory by introducing many more graph theoretic methods (functions) available in SageMath. And we will explore another important type of graph called the multigraph.

Let us begin by recalling an important point we discussed in the second article in this series published in the February 2024 issue of *OSFY*. We constructed a graph from an adjacency list (one way of representing a graph). Afterwards, we printed the adjacency matrix and incidence matrix of the graph we created. Since adjacency matrices and incidence matrices are crucial in understanding graphs, we will begin by constructing the same graphs discussed in that article, first using an adjacency matrix and later using an incidence matrix. Please refer to the article for an explanation of adjacency matrix and incidence matrix of a graph.

Now, we will construct a graph (which we will call *Graph_3*) using an adjacency matrix with SageMath. Take a look at the SageMath code saved as '*graph3.sagews*', provided below (line numbers have been included for clarity in the explanation).

```
1.   from sage.graphs.graph import
Graph
2.   adj_matrix = [
       [0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 1, 0],
       [1, 0, 1, 0, 1, 0],
       [0, 1, 1, 1, 0, 1],
       [1, 0, 0, 0, 1, 0]
     ]
3.   G = Graph( )
4.   for i in range(len(adj_matrix)):
5.      for j in range(i+1, len(adj_
matrix[i])):
6.         if adj_matrix[i][j] == 1:
7.            G.add_edge(i, j)
8.   G.show( )
```

Now, let us go through the code for better understanding. As before, Line 1 imports the Graph class from the *sage.graphs.graph* module in the SageMath library. This class is used to create and manipulate graphs. Line 2 stores the adjacency matrix of the graph into the variable *adj_matrix*. Line 3 creates an instance of the Graph class, initialising an empty graph to the variable *G*. Line 4 contains a *for* loop that iterates over the rows of the adjacency matrix. Line 5 is a nested *for* loop that iterates over the elements of the current row, starting from *i+1*. This is done to avoid adding edges twice, as the